Experiment 1

Aim: To Implement Linear and Logistic Regression on real-world datasets

Linear Regression Objective

Goal: Predict **exact medical insurance cost (charges)** for an individual based on features like age, BMI, smoking status, number of children, sex, and region.

- **Why Linear Regression:**
    - Target variable (charges) is **continuous**
    - Model is **interpretable**, easy to understand which features contribute to cost
    - RMSE and R^2 can quantify **prediction accuracy**

Logistic Regression Objective

Goal: Predict whether an individual is **high-risk** (i.e., likely to have above-average insurance charges).

- **Target variable:** HighRisk = 1 if charges > average, else 0
- **Why Logistic Regression:**
    - Target variable is **binary** (high-risk or normal)
    - Outputs **probabilities**, helpful for decision-making in insurance
    - Evaluation via accuracy, precision, recall, F1-score, and confusion matrix

Dataset Source:
Medical Cost Personal Dataset : https://www.kaggle.com/datasets/mirichoi0218/insurance

age	sex	bmi	children	smoker	region	charges
19	female	27.9	0	yes	southwest	16884.924
18	male	33.77	1	no	southeast	1725.5523
28	male	33	3	no	southeast	4449.462
33	male	22.705	0	no	northwest	21984.471
32	male	28.88	0	no	northwest	3866.8552
31	female	25.74	0	no	southeast	3756.6216
46	female	33.44	1	no	southeast	8240.5896
37	female	27.74	3	no	northwest	7281.5056
37	male	29.83	2	no	northeast	6406.4107
60	female	25.84	0	no	northwest	28923.137
25	male	26.22	0	no	northeast	2721.3208
62	female	26.29	0	yes	southeast	27808.725

Dataset Description:

The **Medical Insurance Cost Dataset** contains personal and lifestyle information of individuals along with their medical insurance charges.

- **Domain:** Healthcare / Insurance analytics
- **Number of instances:** 1,338
- **Type of data:** Structured, tabular, real-world
- **Nature:** Mix of numerical and categorical features

Features

- **age:** Age of the insured person
- **sex:** Gender (male/female)
- **bmi:** Body Mass Index
- **children:** Number of dependents
- **smoker:** Smoking status (yes/no)
- **region:** Residential region
- **charges:** Medical insurance cost (target for regression)

Targets Used

- **Linear Regression:** charges (continuous medical cost)
- **Logistic Regression:** HighRisk
    - 1 → High-cost individual
    - 0 → Normal-cost individual

This dataset is suitable because it represents a **real-life financial and medical decision-making problem**.

Mathematical Formulation of the Algorithm:
1. Linear Regression

Purpose
To predict the **exact medical insurance cost** incurred by an individual.

Model Equation

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

Where:

- $\hat{y}$ = predicted insurance charge
- x_i = input features (age, bmi, smoker, etc.)
- β_i = model parameters

Loss Function

$$MSE = \frac{1}{n} \sum (y - \hat{y})^2$$

Why MSE?

- Penalizes large prediction errors
- Smooth and differentiable
- Suitable for continuous values

2. Logistic Regression

Purpose

To classify whether a person is **high-risk (high cost)** or **not**.

Sigmoid Function

$$P(y = 1|x) = \frac{1}{1 + e^{-(\beta_0 + \beta^T x)}}$$

This converts linear output into a **probability between 0 and 1**.

Loss Function (Log Loss)

$$-\frac{1}{n} \sum [y \log(p) + (1 - y) \log(1 - p)]$$

Why Log Loss?

- Penalizes confident wrong predictions
- Optimizes probability estimates

- Ideal for binary classification

Algorithm Limitations:
Linear Regression Limitations

- Assumes a linear relationship between features and target
- Sensitive to outliers (extreme medical costs)
- Cannot capture complex non-linear patterns
- Multicollinearity affects coefficient stability

Logistic Regression Limitations

- Assumes linear decision boundary
- Performance degrades with highly imbalanced classes
- Cannot model complex interactions without feature engineering
- Less accurate than non-linear classifiers for complex data

Methodology / Workflow:

1. Data Collection
2. Data Cleaning & Encoding
3. Feature Scaling
4. Train–Test Split
5. Model Training
6. Model Evaluation

Linear Regression Workflow

1. Load dataset

Choose Files insurance.csv
insurance.csv(text/csv) - 55628 bytes, last modified: 2/2/2026 - 100% done
```
Saving insurance.csv to insurance.csv
{'insurance.csv':
b'age,sex,bmi,children,smoker,region,charges\r\n19,female,27.9,0,yes,southwest,16884.924\r\n18,male,33.77,1,no,southeast,1725.5523\r\n28,male,33,3,no,s
```

2. Encode categorical variables

```
df_encoded = pd.get_dummies(df, drop_first=True)
df_encoded.head()
```

	age	bmi	children	charges	sex_male	smoker_yes	region_northwest	region_southeast	region_southwest
0	19	27.900	0	16884.92400	False	True	False	False	True
1	18	33.770	1	1725.55230	True	False	False	True	False
2	28	33.000	3	4449.46200	True	False	False	True	False
3	33	22.705	0	21984.47061	True	False	True	False	False
4	32	28.880	0	3866.85520	True	False	True	False	False

3. Select charges as target

```
X_lr = df_encoded.drop("charges", axis=1)
y_lr = df_encoded["charges"]

X_train_lr, X_test_lr, y_train_lr, y_test_lr = train_test_split(
    X_lr, y_lr, test_size=0.2, random_state=42
)
```

4. Scale numerical features

```
scaler = StandardScaler()
X_train_lr = scaler.fit_transform(X_train_lr)
X_test_lr = scaler.transform(X_test_lr)
```

5. Split data into training and testing sets

```
X_lr = df_encoded.drop("charges", axis=1)
y_lr = df_encoded["charges"]

X_train_lr, X_test_lr, y_train_lr, y_test_lr = train_test_split(
    X_lr, y_lr, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train_lr = scaler.fit_transform(X_train_lr)
X_test_lr = scaler.transform(X_test_lr)
```

6. Train Linear Regression model

```
lin_model = LinearRegression()
lin_model.fit(X_train_lr, y_train_lr)

y_pred_lr = lin_model.predict(X_test_lr)
```

7. Evaluate using RMSE and R^2 score

```
mse = mean_squared_error(y_test_lr, y_pred_lr
rmse = np.sqrt(mse)

r2 = r2_score(y_test_lr, y_pred_lr)

print("Linear Regression Performance")
print("RMSE:", rmse)
print("R2 Score:", r2)
```

```
Linear Regression Performance
RMSE: 5796.284659276274
R2 Score: 0.7835929767120722
```

8.

Logistic Regression Workflow

1. Create binary target (HighRisk)

```
mean_charge = df_encoded["charges"].mean()
df_encoded["HighRisk"] = (df_encoded["charges"] > mean_charge).astype(int)
```

2. Encode categorical features

```
X_log = df_encoded.drop(["charges", "HighRisk"], axis=1)
y_log = df_encoded["HighRisk"]
```

3. Scale features

```
scaler = StandardScaler()
X_train_log = scaler.fit_transform(X_train_log)
X_test_log = scaler.transform(X_test_log)
```

4. Split data

```
X_train_log = scaler.fit_transform(X_train_log)
X_test_log = scaler.transform(X_test_log)
```

5. Train Logistic Regression model

```
log_model = LogisticRegression(max_iter=1000)
log_model.fit(X_train_log, y_train_log)

y_pred_log = log_model.predict(X_test_log)
```

6. Evaluate using classification metrics

```
print("Logistic Regression Performance")
print("Accuracy:", accuracy_score(y_test_log, y_pred_log))
print("\nClassification Report:\n", classification_report(y_test_log, y_pred_log))
print("\nConfusion Matrix:\n", confusion_matrix(y_test_log, y_pred_log))
```

```
Logistic Regression Performance
Accuracy: 0.9104477611940298

Classification Report:
               precision    recall  f1-score   support

           0       0.89      1.00      0.94       189
           1       1.00      0.70      0.82        79

    accuracy                           0.91       268
   macro avg       0.94      0.85      0.88       268
weighted avg       0.92      0.91      0.91       268


Confusion Matrix:
 [[189   0]
 [ 24  55]]
```

Performance Analysis:

Linear Regression Metrics

- **RMSE (Root Mean Squared Error):** Measures average prediction error
- **R^2 Score:** Measures how much variance is explained by the model

Interpretation:

- Lower RMSE → better cost prediction
- Higher R^2 → better explanatory power

Logistic Regression Metrics

- **Accuracy:** Overall correctness
- **Precision:** Correctly identified high-risk cases
- **Recall:** Ability to catch all high-risk individuals
- **F1-score:** Balance between precision and recall

Interpretation:

- High recall is important to avoid missing high-risk patients
- F1-score provides balanced performance evaluation

Hyperparameter Tuning:

Linear Regression

- **Ridge Regression (L2 Regularization)**
    - Penalizes large coefficients
    - Reduces overfitting
    - Hyperparameter: alpha

Effect: Improves generalization and stabilizes predictions

```
ridge_params = {'alpha': [0.1, 1, 10, 50]}
ridge_grid = GridSearchCV(Ridge(), ridge_params, cv=5)
ridge_grid.fit(X_train_lr, y_train_lr)

print("Best Alpha:", ridge_grid.best_params_)
```

Logistic Regression

Key hyperparameters:

- **C:** Inverse of regularization strength
- **Penalty:** L1 or L2
- **Solver:** Optimization algorithm

Tuning Method:

- Grid Search with cross-validation

Effect:

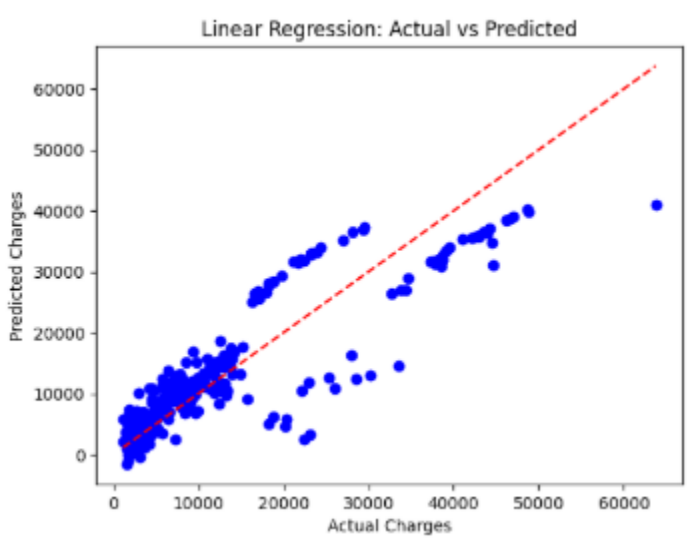
- Improves classification accuracy
- Reduces overfitting
- Produces more reliable probability estimates

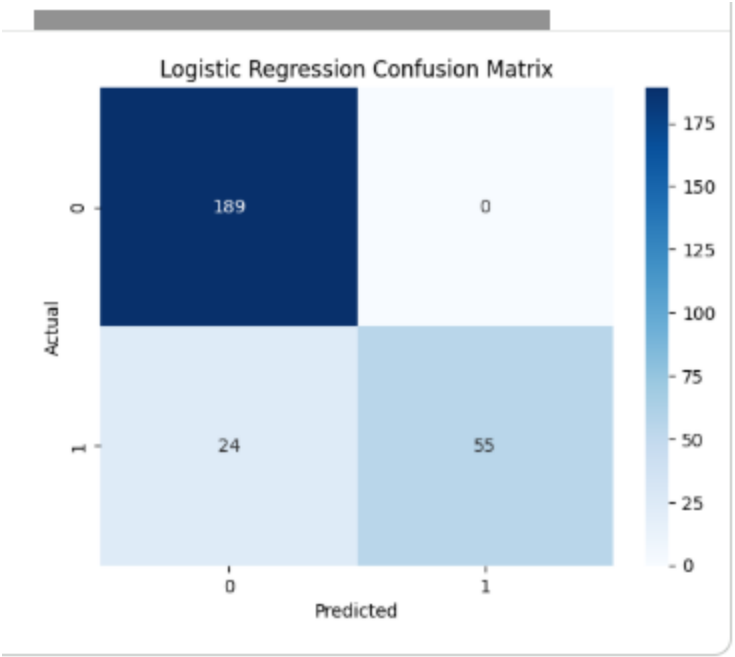
```
param_grid = {
    'C': [0.01, 0.1, 1, 10],
    'penalty': ['l2'],
    'solver': ['lbfgs']
}

grid = GridSearchCV(LogisticRegression(max_iter=1000), param_grid, cv=5)
grid.fit(X_train_log, y_train_log)

print("Best Parameters:", grid.best_params_)
```

```
Best Parameters: {'C': 1, 'penalty': 'l2', 'solver': 'lbfgs'}
```





SDG Goal	How Experiment Relates
3. Good Health & Well-being	Predicts medical cost & identifies high-risk individuals for early intervention
8. Decent Work & Economic Growth	Helps manage financial risk of medical expenses; fair insurance pricing
10. Reduced Inequalities	Identifies vulnerable groups; supports targeted healthcare policies
9. Industry & Innovation	Demonstrates AI/ML application in healthcare analytics